POWC — PHASE 1

Complete Work Report & End-to-End Execution Roadmap

Project focus: Entity-resolution blocking and candidate generation across S1, S2, and S3.

Purpose: Record prior experiments, explain why each step is necessary, and provide a practical checklist to finish Phase 1.

Prepared from the project progress and experiment results discussed so far. Current source code and latest output have not been independently inspected; unverified tasks are explicitly marked.

1. Executive Summary

POWC Phase 1 focuses on candidate generation for entity resolution. Instead of comparing every record from one dataset against every record in another, blocking methods use shared keys to create a smaller set of plausible candidate pairs. The objective is to preserve true matches while controlling candidate volume.

Previously recorded experiments include OR-based blocking (99.59% recall), two-or-more-key blocking (95.73%), adaptive blocking (97.94%), and a combined blocker reported to recover 3,443 of 3,444 known matches (about 99.97%). A global bucket cap of 5,000 was reported to reduce recall to 85.89%, so that approach should not be reused without redesign and validation.

Important: These are historical results from the conversation, not a fresh execution of the current repository. Candidate counts and evaluation definitions must be verified before being treated as final.

Previously recorded experiment	Reported result	Interpretation / status
OR-based blocking	99.59% recall	High recall; candidate volume reported as large. Verify how it was calculated.
Two-or-more-key blocking	95.73% recall	More restrictive; lost more known matches.
Adaptive blocking	97.94% recall	Intermediate recall; evaluate alongside candidate volume.
Combined blocker	3,443 / 3,444 = ~99.97%	Promising; reproduce with the same evaluation set.
Global bucket cap of 5,000	85.89% recall	Previously rejected due to excessive loss of true matches.
Rare address-token rescue	Previously explored/planned	Verify current implementation and measured benefit.
Jupitar / IJPLANNERS	Previously noted as unresolved	Inspect actual records and failure stage.

2. Project Goal and Scope

Goal: Generate candidate pairs across S1–S2 and S1–S3 (and any other required pairing) with high candidate recall and manageable candidate volume.

What Phase 1 does

- Profiles and normalizes source records.
- Builds blocking keys from fields such as names and addresses.
- Generates and deduplicates candidate pairs.
- Measures recall, candidate count, candidate reduction, and missed-match cases.
- Selects and documents a reproducible blocking strategy.

What Phase 1 does not automatically complete

- Final classification of every candidate pair.
- Human review or a deployed entity-resolution service unless explicitly in scope.
- Proof of efficiency from recall alone; candidate-volume evidence is required.

Keep the boundary between blocking and final matching explicit in the report.

3. End-to-End Workflow

Stage	Work to do	Why it matters	Status from discussion
1. Dataset profiling	Record schemas, row counts, IDs, missingness, duplicates for S1/S2/S3.	Establishes actual input structure.	Verify
2. Normalization	Create normalized names/addresses; preserve raw values.	Reduces formatting-related key mismatches.	Verify

Stage	Work to do	Why it matters	Status from discussion
3. Ground truth	Validate known true-match pairs and identifiers.	Provides reference for recall.	3,444 matches previously used; verify source
4. Baselines	Run OR-based and multi-key blockers.	Creates comparable reference points.	Previously tested
5. Adaptive blocking	Use key frequency/characteristics to control candidate generation.	Balances recall and candidate volume.	Previously tested
6. Missed-match analysis	Explain why each known true pair was not generated.	Identifies systematic gaps.	Partial / verify
7. Address-token rescue	Add rare/distinctive address tokens and measure incremental gains.	Recovers cases not covered by name keys.	Verify
8. Volume optimization	Analyze candidate volume by key frequency and source pair.	Reduces unnecessary comparisons.	In progress
9. Combined blocker	Union complementary rules and deduplicate.	Combines strengths of rules.	Previously reported; reproduce
10. Validation	Check duplicates, invalid pair types, self-pairs, missed matches.	Ensures correctness.	Verify
11. Final evaluation	Compare all methods on same known-match set.	Makes results fair and reproducible.	Required
12. Documentation	Record methods, configuration, metrics, limitations, commands.	Makes Phase 1 auditable.	Required

4. Dataset Profiling and Normalization

4.1 Profile each source

- Record file path, number of records, columns, unique identifiers, missingness, and duplicate identifier counts.
- Identify fields used for blocking versus fields used only for evaluation.
- Confirm the meaning and format of ground-truth pair IDs.

Deliverable: dataset profile for S1, S2, and S3, plus a short description of data-quality problems.

4.2 Normalize values

- Normalize text case and whitespace consistently.
- Apply punctuation and address-token rules appropriate to the source fields.
- Handle null/empty values explicitly; do not generate misleading keys from missing values.
- Preserve original values for debugging and later similarity scoring.

Deliverable: repeatable preprocessing script and counts showing missing or changed values.

5. Metrics: How to Evaluate Blocking

5.1 Candidate recall

Candidate Recall = known true matches included in the candidate set / all known true matches in the evaluation set.

With 3,444 known matches, recovering 3,443 yields 3,443 / 3,444 × 100 ≈ 99.97%. This is meaningful only if every method uses the same validated ground-truth set.

5.2 Candidate volume

Count unique candidate pairs after deduplication. Report counts separately for S1–S2 and S1–S3 where applicable. Verify whether historical values described as 'per S1 record' are averages or total pair counts.

5.3 Candidate reduction ratio

Candidate Reduction Ratio = 1 – (unique candidate pairs / all possible cross-dataset pairs). The denominator must reflect the actual source sizes and pairings.

5.4 Additional checks

- Incremental true matches recovered by each rule.
- Incremental candidate pairs introduced by each rule.
- Runtime and peak memory, if relevant.
- Missed true-match list and reason for each miss.
- Candidate-pair duplicate count before and after deduplication.

6. Blocking Strategies and Rationale

6.1 OR-based blocking

A pair is a candidate if any configured blocking key/rule matches. This often preserves recall because different rules capture different variations. Its drawback is potentially large candidate volume.

6.2 Two-or-more-key blocking

A pair is a candidate only when at least two keys match. This can reduce weak or generic candidates but may miss true pairs with name/address variation. Previously recorded recall: 95.73%.

6.3 Adaptive blocking

Adjusts candidate generation based on properties such as key frequency. It attempts to avoid excessive candidate explosion from common keys while retaining useful keys. Previously recorded recall: 97.94%.

6.4 Rare address-token rescue

Adds candidates using distinctive address tokens, potentially recovering true matches with poor name-key overlap. Token length alone does not establish rarity; use token frequency and validate candidate volume.

6.5 Combined blocker

Combines complementary rules and deduplicates candidate pairs. Previously reported result: 3,443 of 3,444 known matches (~99.97%). Re-run before treating it as final.

6.6 Why the global cap was rejected

A global bucket cap of 5,000 was reported to reduce recall to 85.89%. A hard cap can remove valid pairs from large buckets. Prefer frequency-aware rules and evidence-based prioritization; measure the trade-off.

7. Missed-Match Investigation

The Jupitar / IJPLANNERS pair was previously identified as difficult or unresolved. Verify whether it is still missing in the current pipeline.

- Retrieve actual source records and the ground-truth pair.
- Compare raw and normalized names and addresses.
- List blocking keys generated for each record.
- Determine whether no candidate was generated or a candidate was lost downstream.
- Test whether a rare address-token rule recovers it.
- Measure additional candidates caused by the rule.
- Document the reason and decision; avoid undocumented hardcoded exceptions.

Deliverable: a missed-match audit table with pair IDs, failure reason, recovery rule, and result.

8. Candidate-Volume Optimization Plan

Experiment	What to vary	Required measurements
Shared-key count	One shared key vs two or more; preserve rule provenance.	Recall, unique candidates, added/missed matches.
Key frequency	Frequency cutoffs or adaptive policy.	Recall and candidates by frequency band.
Address tokens	Rarity criteria and token eligibility.	Incremental recall, candidates, runtime.
Source pair	S1–S2 vs S1–S3 separately.	Counts and recall per pairing.
Rule combination	Union of selected rules; deduplicate.	Final recall, volume, duplicate count.
Bucket limits	Avoid global hard caps unless justified.	Recall impact at every limit; document rejected policies.

Do not optimize on recall alone. A method generating nearly all possible pairs may have excellent recall but little practical blocking benefit. Select a method using both recall and candidate reduction.

9. Required Final Evaluation

Run every method against the same ground-truth set and source snapshots. Recompute metrics from generated files rather than copying earlier output.

Method	True matches recovered	Recall	Unique candidate pairs	Reduction ratio	Runtime
OR-based	Recompute	99.59% previously reported	Recompute	Recompute	Measure
Two-or-more-key	Recompute	95.73% previously reported	Recompute	Recompute	Measure
Adaptive	Recompute	97.94% previously reported	Recompute	Recompute	Measure
Address-token only	Measure	Measure	Measure	Measure	Measure
Combined blocker	3,443 previously reported	~99.97% previously reported	Recompute	Recompute	Measure

Unverified quantities are marked 'Recompute' or 'Measure'. Do not invent missing results.

10. Correctness and Reproducibility Checklist

- Every candidate pair has valid source record IDs.
- Only allowed source-pair combinations are present.
- No self-pairs or invalid cross-source pairings.
- Candidate pairs are deduplicated after combining rules.
- Ground-truth IDs use the same canonical format as candidate IDs.
- Recall is calculated on the full, fixed evaluation set.
- Candidate volume counts unique pairs, not repeated emissions by rules.
- Raw values are preserved alongside normalized values.
- Final configuration and thresholds are recorded.
- A clean rerun reproduces the reported metrics.

11. Suggested Repository Deliverables

Adapt these names to the actual repository. Do not create duplicates if equivalent working scripts already exist.

Artifact	Purpose
data_profile.md / profile outputs	Schemas, row counts, missingness, source notes.
preprocess.py	Normalization and preparation.
blocking_baselines.py	Baseline OR and multi-key methods.
adaptive_blocking.py	Frequency-aware/adaptive candidate generation.
address_token_blocking.py	Rare address-token candidate rule.
combine_candidates.py	Union and deduplication of candidate pairs.
evaluate_blocking.py	Recall, volume, reduction, and missed-match metrics.
missed_matches.csv	Pair-level audit of unresolved known matches.
experiment_results.csv	One row per method/configuration with measured metrics.
README.md	Reproduction commands, data assumptions, and limitations.
phase1_report.pdf	Final methods, experiments, results, and conclusions.

12. Practical Execution Order

- **Step 1:** Inspect the current repository and identify existing scripts/results; do not repeat completed work.
- **Step 2:** Verify source datasets, ground-truth file, and meaning of the 3,444 known matches.
- **Step 3:** Run existing baseline methods and save outputs.
- **Step 4:** Check whether rare address-token blocking is already implemented; evaluate it if so.
- **Step 5:** Audit Jupitar / IJPLANNERS and other missed matches.
- **Step 6:** Analyze candidate volume by key frequency, shared-key count, and source pair.
- **Step 7:** Combine selected complementary rules and deduplicate pairs.
- **Step 8:** Recompute recall and candidate reduction for every method on the same evaluation set.
- **Step 9:** Run correctness checks and a clean reproducibility test.
- **Step 10:** Freeze the selected configuration and complete the report.

13. Definition of Done for Phase 1

- Source datasets and evaluation set are identified and documented.
- Preprocessing and blocking rules are reproducible.
- All methods use the same known-match set.
- Final candidate recall is recomputed from current outputs.
- Candidate volume and candidate reduction are measured.
- The global-cap experiment and recall loss are documented.
- Remaining missed-match cases are investigated or explicitly reported.
- Candidate pairs pass structural and deduplication checks.
- Final experiment table and README can be regenerated from code.

14. Limitations and Reporting Guidance

- Historical experiment results are not independently verified in this document.
- High recall on a known-match set does not prove all true matches in the population are found.
- The known-match set may be incomplete or biased; document how it was created.

- Candidate recall does not measure final match precision or correctness.
- Candidate volume must have a clear unit and denominator.
- The best configuration depends on the acceptable balance between missed matches and computational cost.

Suggested conclusion: 'The selected Phase 1 blocker achieved [measured recall] on [evaluation set size] known true matches and generated [unique candidate count] candidate pairs, corresponding to [measured candidate reduction]. Remaining missed matches were [count and explanation]. Metrics were reproduced using [configuration/version].' Replace every bracketed field with measured results.

15. Final Recommendation

First inspect the current repository and consolidated experiment outputs. The previously recorded combined recall of approximately 99.97% is promising, but it must be reproduced. The next technical priority is candidate-volume analysis and the missed-match audit, not adding more algorithms without evidence.

A planning estimate is 1–3 focused working days if the existing code is functional and the main blocking methods are already implemented. This is an estimate, not a guarantee.